Finite Field Arithmetic for ML-KEM Using Zech's Logarithm

Masaaki Shirase
Future University Hakodate
shirase@fun.ac.jp

March 2026
Version 1.0

Abstract

The processing of ML-KEM (formerly CRYSTALS-Kyber), a key encapsulation mechanism with post-quantum security, is performed by multiplication, addition, and subtraction of polynomials whose coefficients lie in the finite field $\mathbb{F}_{3329}$. To reduce the number of such operations, it is common to use the Number Theoretic Transform (NTT). This paper focuses on arithmetic over $\mathbb{F}_{3329}$ and proposes the use of a logarithmic representation with respect to a primitive element α of $\mathbb{F}_{3329}^*$ for implementing multiplication, addition, and subtraction over $\mathbb{F}_{3329}$. In this representation, multiplication in $\mathbb{F}_{3329}^*$ can be reduced to addition in $\mathbb{Z}_{3328}$. Furthermore, addition and subtraction in $\mathbb{F}_{3329}^*$ can be computed in the logarithmic domain by using Zech's logarithm. However, special treatment is required when $0 \in \mathbb{F}_{3329}$ is involved in the operations. This paper proposes a new implementation method of the logarithmic representation for arithmetic over $\mathbb{F}_{3329}$, including the handling of such exceptional cases.

Keywords: ML-KEM, Post-quantum cryptography, Zech's logarithm, Finite field arithmetic

1 Introduction

In recent years, with the progress of quantum computing, research on post-quantum cryptographic schemes has become increasingly active. ML-KEM (formerly CRYSTALS-Kyber) [1] is the first key encapsulation mechanism (KEM) selected in the post-quantum cryptography standardization project conducted by the National Institute of Standards and Technology (NIST). The security of ML-KEM is based on the hardness of the module-LWE problem.

ML-KEM is constructed around polynomial arithmetic over the quotient ring $R_p = \mathbb{F}_p[x]/(x^n+1)$, where $(p, n) = (3329, 256)$ are chosen. From an implementation viewpoint, the Number Theoretic Transform (NTT) is commonly used to reduce the number of $\mathbb{F}_p$ multiplications required for polynomial multiplication. In implementations of ML-KEM, Barrett reduction [3] and Montgomery multiplication [6] are frequently used for $\mathbb{F}_p$ multiplication. Indeed, the original paper [1] and the submission to NIST [2] adopt Barrett reduction and Montgomery multiplication for implementing $\mathbb{F}_p$ multiplication, and Barrett reduction is also used in high-speed implementations [5] and hardware implementations [4]. In these reduction methods, the modular reduction modulo p is computed using two multiplications together with additional operations by exploiting precomputed values. These techniques have also been widely used in public-key cryptography even before the advent of post-quantum cryptography.

The Zech's logarithm $Z(x)$ with respect to a primitive element α of the multiplicative group $\mathbb{F}_p^*$ is defined by $\alpha^{Z(x)} = \alpha^x + 1$ [7, 8]. Since addition can be written as $a+b = (a/b+1)\cdot b$, addition can be computed in the logarithmic domain by using Zech's logarithm. Zech's logarithms have been applied to error-correcting codes over finite fields of characteristic two.

A preliminary version of this work was presented in Japanese at SCIS 2026, a domestic symposium on cryptography and information security in Japan.

1.1 Our Contributions

This paper explicitly constructs a new logarithmic representation for multiplication, addition, and subtraction in $\mathbb{F}_{3329}$, incorporating the necessary exceptional handling, and presents the corresponding algorithms, including the necessary treatment of exceptional cases. In this representation, multiplication in $\mathbb{F}_{3329}^*$ is reduced to addition in $\mathbb{Z}_{3328}$, and addition and subtraction in $\mathbb{F}_{3329}^*$ can be computed in the logarithmic domain by using Zech's logarithm. Furthermore, we introduce a Zech's logarithm table dedicated to subtraction. When $0 \in \mathbb{F}_{3329}$ is involved in the operations, special treatment is required.

The table size of Zech's logarithm for $\mathbb{F}_p$ is $O(p \log p)$. Therefore, in conventional public-key cryptosystems such as elliptic curve cryptography, where primes of several hundred bits are used, precomputing such tables is not practical. In contrast, for $\mathbb{F}_{3329}$ used in ML-KEM, the table size fits within the L1 cache (32 KB) of modern CPUs. Hence, the proposed method provides a new design perspective for finite field arithmetic in ML-KEM.

1.2 Notation

We use the following notation throughout this paper:

p	:	a odd prime number
n	:	a positive integer
$\mathbb{Z}_n$	=	$\{0, 1, 2, \ldots, n-1\}$ (residue ring)
$\mathbb{F}_p$	=	$\{0, 1, 2, \ldots, p-1\}$ (finite field)
$\mathbb{F}_p^*$	=	$\{1, 2, \ldots, p-1\}$ (multiplicative group of $\mathbb{F}_p$)
N_p	=	$\{0, 1, 2, \ldots, p-2\} \cup \{-\infty\}$, where $-\infty$ represents the logarithm of 0
$Z(x)$	:	Zech's logarithm
$Z[x]$	:	lookup table implementing Zech's logarithm
$Z_-(x)$	:	Zech's logarithm for subtraction
$Z_-[x]$	:	lookup table implementing $Z_-(x)$
$\widetilde{a}$	:	the element of N_p corresponding to $a \in \mathbb{F}_p$
$\widehat{a}$	:	the bit representation adopted in this paper corresponding to $\widetilde{a} \in N_p$
$\gg 1$	:	right shift by one bit
$(\text{bit string})_2$	:	the unsigned integer obtained by interpreting the bit string as a binary number
$(\text{bit string})_{N_p}$	:	the element of N_p represented by the adopted bit representation

2 Mathematical Preliminaries

The multiplicative group $\mathbb{F}_p^*$ is cyclic, hence there exists an element $\alpha \in \mathbb{F}_p^*$ such that

$$\mathbb{F}_p^* = \{\alpha^i \mid 0 \le i \le p-2\}$$

holds [8]. Such an element α is called a primitive element. By Lagrange's theorem,

$$\alpha^{p-1} = 1$$

holds. Furthermore,

$$\alpha^{(p-1)/2} = -1 \tag{1}$$

holds.

3 Zech's Logarithm

3.1 The Set N_p

For a prime number p, define the set N_p by

$$N_p = \{0, 1, 2, \ldots, p-2\} \cup \{-\infty\}.$$

We define the operation $+$ in N_p as follows:

$$s + t \text{ in } N_p := \begin{cases} s + t \bmod (p-1) & \text{if } s, t \neq -\infty, \\ -\infty & \text{if } s = -\infty \text{ or } t = -\infty. \end{cases} \tag{2}$$

3.2 Zech Logarithm Representation

Fix a primitive element $\alpha \in \mathbb{F}_p^*$ for a finite field $\mathbb{F}_p$. For $a \in \mathbb{F}_p$, we denote by

$$\widetilde{a} = \log_\alpha a$$

the discrete logarithm of a with respect to the base α. By definition,

$$\alpha^{\widetilde{a}} = a$$

holds. We define $\widetilde{0} = -\infty$ and interpret $\alpha^{-\infty} = 0$. Thus, for any $a \in \mathbb{F}_p$, we have $\widetilde{a} \in N_p$.

To emphasize the use of Zech's logarithm (introduced in Section 3.3) for addition and subtraction, we refer to the logarithmic representation expressing $a \in \mathbb{F}_p$ by $\widetilde{a}$ as the **Zech logarithm representation**.

3.3 Zech's Logarithm

Fix a primitive element $\alpha \in \mathbb{F}_p^*$ for a finite field $\mathbb{F}_p$. Define a mapping

$$Z : N_p \to N_p$$

by

$$\alpha^{Z(x)} = \alpha^x + 1. \tag{3}$$

Note that $Z((p-1)/2) = -\infty$ and $Z(-\infty) = 0$. The mapping $Z(x)$ is called **Zech's logarithm** [7].

3.4 Arithmetic in the Zech Logarithm Representation

In this subsection, we describe multiplication, addition, and subtraction over $\mathbb{F}_p$ in the Zech logarithm representation.

Proposition 1 (Multiplication in the Zech Logarithm Representation)**.** *For $a, b \in \mathbb{F}_p$,*

$$\widetilde{a \cdot b} = \widetilde{a} + \widetilde{b} \quad \textit{in } N_p$$

holds (including the cases $a = 0$ or $b = 0$).

Proof. If $a \neq 0$ and $b \neq 0$, then

$$\alpha^{\widetilde{a \cdot b}} = a \cdot b = \alpha^{\widetilde{a}} \cdot \alpha^{\widetilde{b}} = \alpha^{\widetilde{a}+\widetilde{b}},$$

which proves the claim. If $a = 0$, then $a \cdot b = 0$ and $\widetilde{a} = -\infty$. Hence,

$$\alpha^{\widetilde{a \cdot b}} = a \cdot b = 0 = 0 \cdot b = \alpha^{\widetilde{a}} \cdot \alpha^{\widetilde{b}} = \alpha^{\widetilde{a}+\widetilde{b}},$$

which implies that the claim holds. The case $b = 0$ is analogous. □

Proposition 2 (Addition in the Zech Logarithm Representation)**.** *For $a, b \in \mathbb{F}_p$, the value $\widetilde{a+b}$ can be computed by either (a) or (b) below. Note that these computations are performed in N_p.*

$$(a)\begin{cases} Z(\widetilde{a}-\widetilde{b})+\widetilde{b} & \text{if } \widetilde{b} \neq -\infty, \\ \widetilde{a} & \text{if } \widetilde{b} = -\infty, \end{cases}$$
$$(b)\begin{cases} \widetilde{a}+Z(\widetilde{b}-\widetilde{a}) & \text{if } \widetilde{a} \neq -\infty, \\ \widetilde{b} & \text{if } \widetilde{a} = -\infty. \end{cases}$$

Proof. (a) If $\widetilde{b} \neq -\infty$ (equivalently $b \neq 0$), then

$$\begin{aligned} \alpha^{\widetilde{a+b}} &= a+b \\ &= (a/b+1)\cdot b \\ &= (\alpha^{\widetilde{a}}/\alpha^{\widetilde{b}}+1)\cdot\alpha^{\widetilde{b}} \\ &= (\alpha^{\widetilde{a}-\widetilde{b}}+1)\cdot\alpha^{\widetilde{b}} \\ &= \alpha^{Z(\widetilde{a}-\widetilde{b})}\cdot\alpha^{\widetilde{b}} \qquad \text{by (3)} \\ &= \alpha^{Z(\widetilde{a}-\widetilde{b})+\widetilde{b}}. \end{aligned} \tag{4}$$

Hence,

$$\widetilde{a+b} = Z(\widetilde{a}-\widetilde{b})+\widetilde{b}.$$

If $b = 0$, division by 0 occurs in (4), and the above derivation does not apply.

If $\widetilde{b} = -\infty$ (equivalently $b = 0$), then

$$\alpha^{\widetilde{a+b}} = a+b = a = \alpha^{\widetilde{a}},$$

and therefore

$$\widetilde{a+b} = \widetilde{a}.$$

(b) Interchanging a and b in (a) yields (b). □

Before treating subtraction, we give the following lemma.

Lemma 3. *Let α be a primitive element of $\mathbb{F}_p^*$. For any $a \in \mathbb{F}_p$, we have*

$$\widetilde{-a} = \begin{cases} (p-1)/2+\widetilde{a} & \text{if } \widetilde{a} \neq -\infty, \\ -\infty & \text{if } \widetilde{a} = -\infty. \end{cases}$$

Proof. If $\widetilde{a} \neq -\infty$, then

$$\begin{aligned} \alpha^{\widetilde{-a}} &= -a \\ &= -1\cdot a \\ &= \alpha^{(p-1)/2}\cdot\alpha^{\widetilde{a}} \quad \text{by (1)} \\ &= \alpha^{(p-1)/2+\widetilde{a}}. \end{aligned}$$

Hence the lemma holds. If $\widetilde{a} = -\infty$, then $a = 0$, $-a = 0$, and $\widetilde{-a} = -\infty$. □

Proposition 4 (Subtraction in the Zech Logarithm Representation)**.** *For $a, b \in \mathbb{F}_p$, let $h = (p-1)/2$. Then $\widetilde{a-b}$ can be computed by either (a) or (b) below. Note that these computations are performed in N_p.*

$$(a)\begin{cases} Z(\widetilde{a}+h-\widetilde{b})+h+\widetilde{b} & \text{if } \widetilde{b} \neq -\infty, \\ \widetilde{a} & \text{if } \widetilde{b} = -\infty, \end{cases}$$
$$(b)\begin{cases} \widetilde{a}+Z(\widetilde{b}+h-\widetilde{a}) & \text{if } \widetilde{a} \neq -\infty, \\ \widetilde{b}+h & \text{if } \widetilde{a} = -\infty. \end{cases}$$

Proof. Replace $\widetilde{b}$ with $\widetilde{-b}$ in Proposition 2, and apply Lemma 3. Note that

$$-h \equiv h \pmod{(p-1)}.$$

□

3.5 A Small Example – Zech Logarithm Representation over $\mathbb{F}_7$

The element $3 \in \mathbb{F}_7^*$ is a primitive element. For $a \in \mathbb{F}_7$, the corresponding $\widetilde{a} \in N_7$ is given in Table 1.

Table 1: Correspondence between $a \in \mathbb{F}_7$ and $\widetilde{a} \in N_7$

$a \in \mathbb{F}_7$	$\leftrightarrow$	$\widetilde{a} \in N_7$
0	$\leftrightarrow$	$-\infty$
1	$\leftrightarrow$	0
2	$\leftrightarrow$	2
3	$\leftrightarrow$	1
4	$\leftrightarrow$	4
5	$\leftrightarrow$	5
6	$\leftrightarrow$	3

The Zech's logarithm $Z(s)$ for $s \in N_7$ is given in Table 2. For reference, the corresponding element $3^s \in \mathbb{F}_7$ is also listed.

Table 2: Values of $Z(s)$ for $s \in N_7$

$s \in N_7$		$Z(s) \in N_7$		$3^s \in \mathbb{F}_7$
0	$\leftrightarrow$	2	$\leftrightarrow$	1
1	$\leftrightarrow$	4	$\leftrightarrow$	3
2	$\leftrightarrow$	1	$\leftrightarrow$	2
3	$\leftrightarrow$	$-\infty$	$\leftrightarrow$	6
4	$\leftrightarrow$	5	$\leftrightarrow$	4
5	$\leftrightarrow$	3	$\leftrightarrow$	5
$-\infty$	$\leftrightarrow$	0	$\leftrightarrow$	0

We compute the additions and subtraction $3+5=1$, $3+4=0$, $3+0=3$, and $3-4=6$ using the Zech logarithm representation. All computations below are performed in N_7. When $-\infty$ is not involved, the addition is taken modulo 6 as defined in (2).

(i) In the Zech logarithm representation, $3+5$ is computed as follows.

$$\begin{aligned}
\widetilde{3+5} &= Z(\widetilde{3}-\widetilde{5})+\widetilde{5} && \text{by Proposition 2(a)} \\
&= Z(1-5)+5 && \text{by Table 1} \\
&= Z(-4)+5 \\
&= Z(2)+5 \\
&= 1+5 && \text{by Table 2} \\
&= 6 \\
&= 0
\end{aligned}$$

Therefore, $3+5=3^0=1$, and the computation is correctly performed in the Zech logarithm representation.

(ii) In the Zech logarithm representation, $3+4$ is computed as follows.

$$\begin{aligned}
\widetilde{3+4} &= Z(\widetilde{3}-\widetilde{4})+\widetilde{4} && \text{by Proposition 2(a)}\\
&= Z(1-4)+4 && \text{by Table 1}\\
&= Z(-3)+4\\
&= Z(3)+4\\
&= -\infty+4 && \text{by Table 2}\\
&= -\infty
\end{aligned}$$

Therefore, $3+4=3^{-\infty}=0$, and the computation is correctly performed in the Zech logarithm representation.
(iii) In the Zech logarithm representation, $3+0$ is computed as follows.

$$\begin{aligned}
\widetilde{3+0} &= \widetilde{3} && \text{by Proposition 2(a)}\\
&= 1 && \text{by Table 1}
\end{aligned}$$

Hence, $3+0=3^1=3$, and the computation is correctly performed in the Zech logarithm representation.
(iv) In the Zech logarithm representation, $3-4$ is computed as follows.

$$\begin{aligned}
\widetilde{3-4} &= \widetilde{3}+Z(\widetilde{4}+3-\widetilde{3}) && \text{by Proposition 4(b)}\\
&= 1+Z(4+3-1) && \text{by Table 1}\\
&= 1+Z(0)\\
&= 1+2 && \text{by Table 2}\\
&= 3
\end{aligned}$$

Therefore, $3-4=3^3=6$, and the computation is correctly performed in the Zech logarithm representation.

4 Proposed Method

4.1 Motivation of the Proposed Method

In the Zech logarithm representation, elements of $\mathbb{F}_p$ are represented as elements of N_p. However, since $-\infty \in N_p$ cannot be treated as an ordinary "number,"special care is required for implementation. This section proposes an example of a bit representation that takes into account the implementation of addition and subtraction in the Zech logarithm representation, and presents multiplication, addition, and subtraction algorithms based on this representation. Furthermore, to reduce the computational cost of subtraction, we propose a subtraction-oriented Zech's logarithm. To suppress this additional cost, we propose a subtraction-oriented Zech's logarithm.

In this section, we focus on the case $p=3329$, which is used in ML-KEM.

4.2 An Example of a Bit Representation for the Zech Logarithm Representation

An element of $\mathbb{Z}_{3328}$ (which is a subset of N_{3329}) can be represented using 12 bits. In the bit representation proposed in this paper, an additional LSB, denoted by z, is appended to these 12 bits, so that an element of N_{3329} is represented using 13 bits in total. The bit z is a flag used to distinguish $-\infty$ (i.e., the logarithm of 0). This representation is denoted, with the subscript Np, by

$$(x_{11}\, x_{10}\, \cdots\, x_0\, z)_{Np}, \tag{5}$$

where $x_i \in \{0,1\}$.

If $z = 0$, then $(x_{11}\, x_{10} \cdots x_0)_2 \in \mathbb{Z}_{3328} (\subset N_{3329})$ is represented.

If $z = 1$, the value represents $-\infty$, regardless of the values of x_i. For implementation convenience, we assume

$$0 \le (x_{11}\, x_{10} \cdots x_0)_2 \le 3327 (= p - 2).$$

Equivalently,

$$0 \le (x_{11}\, x_{10} \cdots x_0\, z)_2 \le 6655 (= 2p - 3). \tag{6}$$

For example,

$$(1\ \underbrace{0\,0\,0\,0\,0\,0\,0\,0\,0\,0}_{10\text{ zeros}}\ 1\,0)_{Np}$$

represents $2049 \in N_{3329}$, while

$$(1\ \underbrace{0\,0\,0\,0\,0\,0\,0\,0\,0\,0}_{10\text{ zeros}}\ 1\,1)_{Np}$$

represents $-\infty \in N_{3329}$. Since one element of N_{3329} is represented by 13 bits, it can be implemented using an unsigned 2-byte integer.

We denote by $\widehat{a}$ the bit representation of $\widetilde{a} \in N_p$ in the form of (5).

4.3 An Example Implementation of Multiplication, Addition, and Subtraction Using the Bit Representation

In this subsection, we present an example implementation of multiplication, addition, and subtraction in N_{3329} using the proposed bit representation for the Zech logarithm representation.

For addition and subtraction, the Zech's logarithm $Z : N_{3329} \to N_{3329}$ is required. To implement the Zech's logarithm, a lookup table $Z[x]$ is prepared. The index x of $Z[x]$ is taken from N_{3329}. Although it is possible to use the bit representation directly as the table index, this would approximately double the table size. While this approach reduces the need for exception handling, it increases memory usage. Therefore, we do not adopt this approach.

There are two possible policies for handling the index corresponding to $-\infty \in N_{3329}$. One approach is to perform explicit exception handling whenever $Z(-\infty)$ is required. The other approach is to assign a dedicated index (e.g., $p - 1$) to represent $-\infty$. In this paper, we adopt the former approach, and assume that the index x always satisfies $x \in \mathbb{Z}_{3328}$.

To obtain the index x of the table $Z[x]$ from the bit representation $(x_{11} \cdots x_0\, z)_{Np}$, a one-bit right shift is required to remove the LSB. In this subsection, for a variable w appearing in an algorithm, we denote by $w^{(n)}$ the value of w after completing Step n. For example, Step 2 of Algorithm 1 is

$$2.\ t_0 \leftarrow t_0\ \&\ 0x0001,$$

and $t_0^{(2)}$ denotes the result of $t_0\ \&\ 0x0001$.

All additions and subtractions appearing in the algorithms of this subsection are ordinary integer additions and subtractions.

Proposition 5. *Algorithm 1 outputs the bit representation* $\widehat{a \cdot b}$ *in* N_{3329}.

Algorithm 1 : *Multiplication*
Input: $\widehat{a}, \widehat{b}$ *(bit representations in* N_{3329}*)*
Output: $\widehat{a \cdot b}$ *(bit representation in* N_{3329}*)*
1. $t_0 \leftarrow \widehat{a} \mid \widehat{b}$
2. $t_0 \leftarrow t_0\ \&\ 0x0001$
3. $a_0 \leftarrow \widehat{a}\ \&\ 0xfffe$
4. $b_0 \leftarrow \widehat{b}\ \&\ 0xfffe$
5. $t \leftarrow a_0 + b_0$
6. ***if*** $t \ge 6656$ ***then*** $t \leftarrow t - 6656$
7. $t \leftarrow t \mid t_0$
8. ***return*** t

Proof. Let $p = 3329$. By Proposition 1, it suffices to show that the output of Algorithm 1 is the bit representation corresponding to $\widetilde{a} + \widetilde{b}$. Let

$$\widehat{a} = (a_{11} \cdots a_0\, z_a)_{Np}, \quad \widehat{b} = (b_{11} \cdots b_0\, z_b)_{Np}.$$

We first show that

$$0 \le (t^{(7)})_2 \le 6655. \tag{7}$$

Since $\widehat{a}$ and $\widehat{b}$ satisfy condition (6), we have

$$0 \le (a_0^{(3)})_2,\ (b_0^{(4)})_2 \le 6654,$$

and therefore

$$0 \le (t^{(5)})_2 \le 13308.$$

After Step 6, $(t^{(6)})_2$ is even and satisfies

$$0 \le (t^{(6)})_2 \le 6654.$$

Hence (7) holds.
(i) Case $z_a = z_b = 0$: In this case, $\widehat{a}$ corresponds to $(a_{11} \cdots a_0)_2$, and $\widehat{b}$ corresponds to $(b_{11} \cdots b_0)_2$. The LSB of $t_0^{(1)}$ is 0, hence $t_0^{(2)} = 0$. Therefore, $t^{(7)}$ satisfies (7) and

$$(t^{(7)})_2 \equiv 2\widehat{a} + 2\widehat{b} \pmod{6656}.$$

Thus $t^{(7)}$ is the bit representation corresponding to $\widetilde{a} + \widetilde{b}$.
(ii) Case $z_a = 1$: Then $\widehat{a}$ corresponds to $-\infty$. The LSB of $t_0^{(1)}$ is 1, so $t_0^{(2)} = 1$. Since $t^{(7)}$ satisfies (7) and its LSB is 1, $t^{(7)}$ represents $-\infty$, which equals $\widehat{a} + \widehat{b}$.
(iii) Case $z_b = 1$: This case is analogous to (ii). □

Proposition 6. *Algorithm 2 outputs the bit representation* $\widehat{a+b}$ *in* N_{3329}.

Algorithm 2 : *Addition*
Input: $\widehat{a}, \widehat{b}$ *(bit representations in* N_{3329}*)*
Output: $\widehat{a+b}$ *(bit representation in* N_{3329}*)*
1. $s \leftarrow \widehat{a} + 6656$
2. $s \leftarrow s - \widehat{b}$
3. ***if*** $s \ge 6656$ ***then*** $s \leftarrow s - 6656$
4. $s_1 \leftarrow s \gg 1$
5. $t \leftarrow Z[s_1] + \widehat{b}$
6. ***if*** $t \ge 6656$ ***then*** $t \leftarrow t - 6656$
7. ***if*** $(\widehat{a}\ \&\ 0x0001) = 0x0001$ ***then*** $t \leftarrow \widehat{b}$
8. ***else if*** $(\widehat{b}\ \&\ 0x0001) = 0x0001$ ***then*** $t \leftarrow \widehat{a}$
9. ***return*** t

Proof. Let $p = 3329$. We show that Algorithm 2 computes Proposition 2(a) in the bit representation. Let

$$\widehat{a} = (a_{11} \cdots a_0\, z_a)_{Np}, \quad \widehat{b} = (b_{11} \cdots b_0\, z_b)_{Np}.$$

For s and s_1, we obtain

$$\begin{aligned} &0 \le s^{(2)} \le 13311, \\ &0 \le s^{(3)} \le 6655, \\ &0 \le s_1^{(4)} \le 3327. \end{aligned} \tag{8}$$

Hence $s_1^{(4)}$ is a valid index of $Z[x]$. Moreover,

$$\begin{aligned} 0 \le t^{(5)} \le 13310, \\ 0 \le t^{(6)} \le 6655. \end{aligned} \tag{9}$$

(i) Case $z_a = z_b = 0$: From (8),

$$(s_1^{(4)})_2 = (a_{11} \cdots a_0)_2 - (b_{11} \cdots b_0)_2 \pmod{3328}.$$

From (9),

$$(t^{(6)})_2 = Z[s_1^{(4)}] + \widehat{b} \pmod{6656}.$$

Since Steps 7 and 8 are not executed, the output of Algorithm 2 is the bit representation of $Z(\widetilde{a} - \widetilde{b}) + \widetilde{b}$.
(ii) Case $z_a = 1$ ($\Leftrightarrow \widetilde{a} = -\infty$): Note that

$$Z(\widetilde{a} - \widetilde{b}) + \widetilde{b} = Z(-\infty) + \widetilde{b} = 0 + \widetilde{b} = \widetilde{b}.$$

Step 7 is executed, and the bit representation $\widehat{b}$ is returned.
(iii) Case $z_b = 1$: Step 8 is executed, and the bit representation $\widehat{a}$ is returned. □

Proposition 7. *Algorithm 3 outputs the bit representation* $\widehat{a - b}$ *in* N_{3329}.

Algorithm 3 : *Subtraction*	
Input: $\widehat{a}, \widehat{b}$ *(bit representations in* N_{3329}*)*	
Output: $\widehat{a - b}$ *(bit representations in* N_{3329}*)*	
1.	$c \leftarrow \widehat{b} + 3328$
2.	***if*** $c \ge 6656$ ***then*** $c \leftarrow c - 6656$
3.	$s \leftarrow c + 6656$
4.	$s \leftarrow s - \widehat{a}$
5.	***if*** $s \ge 6656$ ***then*** $s \leftarrow s - 6656$
6.	$s_1 \leftarrow s \gg 1$
7.	$t \leftarrow \widehat{a} + Z[s_1]$
8.	***if*** $t \ge 6656$ ***then*** $t \leftarrow t - 6656$
9.	***if*** $(\widehat{b}$ & $0x0001) = 0x0001$ ***then*** $t \leftarrow \widehat{a}$
10.	***else if*** $(\widehat{a}$ & $0x0001) = 0x0001$ ***then*** $t \leftarrow c$
11.	***return*** t

Proof. Let $p = 3329$ and $h = (p-1)/2 = 1664$. We show that Algorithm 3 computes Proposition 4(b) in the bit representation. Let

$$\widehat{a} = (a_{11} \cdots a_0\, z_a)_{Np}, \quad \widehat{b} = (b_{11} \cdots b_0\, z_b)_{Np}.$$

Observe that

$$c = 2h + \widehat{b} \bmod 6656.$$

For s and s_1, we obtain

$$\begin{aligned} 0 \le s^{(4)} \le 13311, \\ 0 \le s^{(5)} \le 6655, \\ 0 \le s_1^{(6)} \le 3327. \end{aligned} \tag{10}$$

Hence $s_1^{(6)}$ is a valid index of $Z[x]$. Moreover,

$$\begin{aligned} 0 \le t^{(7)} \le 13310, \\ 0 \le t^{(8)} \le 6654. \end{aligned} \tag{11}$$

(i) Case $z_a = z_b = 0$: From (10),

$$(s_1^{(6)})_2 = (b_{11} \cdots b_0)_2 + h - (a_{11} \cdots a_0)_2 \bmod 3328.$$

From (11),

$$(t^{(8)})_2 = \widehat{a} + Z[s_1^{(6)}] \bmod 6656.$$

Since Steps 9 and 10 are not executed, the output of Algorithm 3 is the bit representation of

$$\widetilde{a} + Z(\widetilde{b} + h - \widetilde{a}),$$

which equals $\widetilde{a - b}$.
(ii) Case $z_b = 1 (\Leftrightarrow \widetilde{b} = -\infty)$: We have

$$\widetilde{a} + Z(\widetilde{b} + h - \widetilde{a}) = \widetilde{a} + Z(-\infty) = \widetilde{a} + 0 = \widetilde{a}.$$

Step 9 is executed, and $\widehat{a}$ is returned.
(iii) Case $z_a = 1$ ($\Leftrightarrow \widetilde{a} = -\infty$): Step 10 is executed, and $c^{(2)}$, which corresponds to $h + \widetilde{b}$, is returned. □

Algorithm 3 contains two more conditional branches than Algorithm 2, and also requires several additional additions. This implies that, under the proposed bit representation, subtraction is computationally more expensive than addition.

4.4 Subtraction-Oriented Zech's Logarithm

Let $\mathbb{F}_p$ be a finite field, and let $\alpha \in \mathbb{F}_p^*$ be a primitive element. Define the mapping (subtraction-oriented Zech's logarithm) $Z_- : N_p \to N_p$ by

$$\alpha^{Z_-(x)} = \alpha^x - 1. \tag{12}$$

Note that

$$Z_-(0) = -\infty, \qquad Z_-(-\infty) = (p-1)/2.$$

Let $Z_-[x]$ denote the lookup table for the subtraction-oriented Zech's logarithm $Z_- : N_{3329} \to N_{3329}$. The index x of the table is taken from N_{3329}.

Proposition 8 (Subtraction Using $Z_-(x)$)**.** *For $a, b \in \mathbb{F}_p$, $\widetilde{a - b}$ can be computed in N_p as follows:*

$$\begin{cases} Z_-(\widetilde{a} - \widetilde{b}) + \widetilde{b} & \text{if } \widetilde{b} \neq -\infty, \\ \widetilde{a} & \text{if } \widetilde{b} = -\infty. \end{cases}$$

Proof. If $\widetilde{b} \neq -\infty$ ($\Leftrightarrow b \neq 0$), then

$$\begin{aligned} \alpha^{\widetilde{a-b}} &= a - b \\ &= (a/b - 1) \cdot b \qquad (13) \\ &= (\alpha^{\widetilde{a}}/\alpha^{\widetilde{b}} - 1) \cdot \alpha^{\widetilde{b}} \\ &= (\alpha^{\widetilde{a}-\widetilde{b}} - 1) \cdot \alpha^{\widetilde{b}} \\ &= \alpha^{Z_-(\widetilde{a}-\widetilde{b})} \cdot \alpha^{\widetilde{b}} \qquad \text{by (12)} \\ &= \alpha^{Z_-(\widetilde{a}-\widetilde{b})+\widetilde{b}}. \qquad (14) \end{aligned}$$

Hence

$$\widetilde{a-b} = Z_-(\widetilde{a}-\widetilde{b})+\widetilde{b}.$$

If $b=0$, division by 0 occurs in (13), and the above derivation does not apply.

If $\widetilde{b}=-\infty$ (equivalently $b=0$), then

$$\alpha^{\widetilde{a-b}} = a-b = a = \alpha^{\widetilde{a}},$$

and therefore $\widetilde{a-b}=\widetilde{a}$. □

Proposition 8 has the same structure as Proposition 2(a), except that $Z_-(x)$ is used instead of $Z(x)$.

Proposition 9. *Algorithm 4 outputs the bit representation* $\widehat{a-b}$ *in* N_{3329}.

Algorithm 4 : *Subtraction Using the Subtraction-Oriented Zech's Logarithm*

Input: $\widehat{a},\widehat{b}$ *(bit representations in* N_{3329}*)*

Output: $\widehat{a-b}$ *(bit representation in* N_{3329}*)*

1. $s \leftarrow \widehat{a}+6656$
2. $s \leftarrow s-\widehat{b}$
3. ***if*** $s \geq 6656$ ***then*** $s \leftarrow s-6656$
4. $s_1 \leftarrow s \gg 1$
5. $t \leftarrow Z_-[s_1]+\widehat{b}$
6. ***if*** $t \geq 6656$ ***then*** $t \leftarrow t-6656$
7. ***if*** $(\widehat{a}\ \&\ 0x0001) = 0x0001$ ***then***
8. $\quad t \leftarrow 3328+\widehat{b}$
9. $\quad$ ***if*** $t \geq 6656$ ***then*** $t \leftarrow t-6656$
10. ***else if*** $(\widehat{b}\ \&\ 0x0001) = 0x0001$ ***then*** $t \leftarrow \widehat{a}$
11. ***return*** t

Proof. The proof is analogous to that of Proposition 6. When $z_a = 1$ ($\Leftrightarrow \widetilde{a} = -\infty$), the bit representation of $\widetilde{-b}$ must be returned. Therefore, unlike Algorithm 2, Steps 8 and 9 are required. □

Comparing Algorithm 2 and Algorithm 4, Algorithm 4 replaces $Z[x]$ with $Z_-[x]$, and introduces the additional operation "3328+" in Step 8 together with the conditional reduction in Step 9.

4.5 Discussion

The table $Z[x]$ consists of 3,328 entries, each of which is a 2-byte value. Hence its size is 6,656 bytes. The table $Z_-[x]$ has the same size, namely 6,656 bytes. When the bit representation of N_p is used for arithmetic in $\mathbb{F}_{3329}$, a conversion table from elements of $\mathbb{F}_{3329}$ to their bit representations may be required, as well as the inverse conversion table. Each of these tables contains $3,329 \times 2 = 6,658$ bytes. The total size of these four tables is approximately 26 KB, which fits within the typical size of the L1 data cache (32 KB) of modern CPUs. If Algorithm 3 is used for subtraction, the computation time increases, but the table $Z_-[x]$ becomes unnecessary.

In general, the computational cost of arithmetic in $\mathbb{F}_p$ satisfies

$$\mathbb{F}_p\text{-multiplication} \gg \mathbb{F}_p\text{-subtraction} \geq \mathbb{F}_p\text{-addition}.$$

However, when arithmetic in $\mathbb{F}_{3329}$ is performed using the proposed bit representation, the cost relation becomes

$$\mathbb{F}_p\text{-subtraction} \geq \mathbb{F}_p\text{-addition} > \mathbb{F}_p\text{-multiplication}.$$

5 Conclusion and Future Work

In this paper, we proposed the use of the Zech logarithm representation for arithmetic in $\mathbb{F}_{3329}$ used in ML-KEM. We introduced a bit representation to implement the Zech logarithm representation and presented algorithms for multiplication, addition, and subtraction in $\mathbb{F}_{3329}$ under this bit representation. The Zech logarithm representation requires lookup tables such as $Z[x]$. However, when $p = 3329$, the total table size fits within the L1 data cache (32 KB). A distinctive feature of the Zech logarithm representation is that the computational cost of arithmetic becomes

$$\mathbb{F}_p\text{-subtraction} \geq \mathbb{F}_p\text{-addition} > \mathbb{F}_p\text{-multiplication}.$$

Although the treatment of $-\infty \in N_p$ requires special handling, multiplication is reduced to integer addition, and addition and subtraction are reduced to table access and integer addition/subtraction.

The algorithms for $\mathbb{F}_{3329}$ arithmetic under the proposed bit representation (Algorithms 1–4) should be regarded as examples, and further optimization remains possible. In future work, we plan to evaluate the implementation cost of the proposed arithmetic, as well as to investigate hardware implementations.

References

[1] Bos, Joppe, et al. "CRYSTALS-Kyber: a CCA-secure module-lattice-based KEM." 2018 IEEE European symposium on security and privacy (EuroS&P). IEEE, 2018.

[2] Avanzi, Roberto, et al. "CRYSTALS-Kyber algorithm specifications and supporting documentation." NIST PQC Round 2.4 (2019): 1-43.

[3] Barrett, Paul. "Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor." Conference on the Theory and Application of Cryptographic Techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 1986.

[4] Huang, Yiming, et al. "A pure hardware implementation of CRYSTALS-KYBER PQC algorithm through resource reuse." IEICE Electronics Express 17.17 (2020): 1-6.

[5] Abdulrahman, Amin, et al. "Faster kyber and dilithium on the cortex-m4." International Conference on Applied Cryptography and Network Security. Cham: Springer International Publishing, 2022.

[6] Montgomery, Peter L. "Modular multiplication without trial division." Mathematics of computation 44.170 (1985): 519-521.

[7] Huber, Klaus. "Some comments on Zech's logarithms." IEEE Transactions on Information Theory 36.4 (2002): 946-950.

[8] Lidl, Rudolf, and Harald Niederreiter. *Introduction to finite fields and their applications.* Cambridge university press, 1994.